

SHIBBOLETH: A Generative Multilingual Evaluation Framework for Adversarial Prompt Classifiers

Octave Commons

Draft initialized from repository specs and research notes

April 4, 2026

Abstract

We present SHIBBOLETH, a generative and reproducible evaluation framework for multilingual adversarial prompt classification. Existing prompt safety datasets are often English-centric, template-heavy, and evaluated as static artifacts rather than living benchmark grammars. SHIBBOLETH reframes the problem around two coupled failure modes: safety bypass, where adversarial prompts pass through to the target model, and availability degradation, where prompts induce refusal spirals, latency inflation, or token burn without a successful compromise. The framework combines a deterministic seven-stage data pipeline, split-before-transform leakage prevention, multilingual and obfuscation robustness suites, and Security–Utility–Efficiency tradeoff analysis. Our central claim is that the grammar of benchmark generation, provenance, and evaluation matters as much as the final released rows. This draft captures the paper boundary around that claim while leaving empirical tables to the evaluation harness as it stabilizes.

1 Introduction

Large language models deployed behind toolchains and policy routers face an adversarial input problem that is broader than classic jailbreak detection. Attackers may seek harmful outputs, prompt leakage, tool abuse, or policy mapping, but they may also degrade system usefulness through repeated refusals, latency inflation, and token exhaustion. In practice, these attacks are multilingual, iterative, and often obfuscated by code-mixing or Unicode tricks.

Current evaluation practice leaves several gaps. First, many prompt safety corpora are effectively English-only or rely on post hoc translation rather than native adversarial coverage [2, 5, 6]. Second, dataset construction often permits semantic leakage when translation and augmentation occur before split assignment. Third, evaluation reports tend to collapse the problem into safety-only metrics while underreporting utility and efficiency costs. These gaps make it difficult to defend front-door classifier claims in production settings where multilingual traffic and operational overhead both matter.

SHIBBOLETH is intended as a benchmark grammar rather than a one-shot dataset snapshot. The central claim is that adversarial prompt classifier evaluation should be built around reproducible generation rules, provenance-preserving dataset construction, and explicit tradeoff measurement across safety, utility, and efficiency. The resulting benchmark can regenerate as attack styles evolve while preserving stable receipts about how each artifact was built.

This paper makes five concrete contributions.

1. It defines a production-facing threat model for multilingual adversarial prompt classification with both safety and availability failure modes.

2. It introduces a deterministic build pipeline with cluster-level split-before-transform invariants that aim to prevent cross-lingual leakage.
3. It frames multilingual and obfuscation transforms as governed benchmark operators rather than ad hoc augmentation scripts.
4. It organizes evaluation into direct attack, benign lookalike, obfuscation, and resource-exhaustion partitions.
5. It proposes Security–Utility–Efficiency tradeoff analysis as the right reporting surface for front-door prompt classification.

The rest of the paper develops these claims from the current repository design. The intended paper identity is therefore a benchmark and evaluation paper, not a claim that one classifier architecture has already won the problem.

2 Related Work

Adversarial prompt evaluation currently sits at the intersection of jailbreak datasets, multilingual safety corpora, and broader dataset-governance practice. Public resources such as All-Prompt-Jailbreak, WildJailbreak, Aya RedTeaming, and XSafety are useful starting points because they make attack prompts easier to compare across systems, but they also illustrate the fragmentation of the space: some are template-heavy, some are multilingual without deep native adversarial coverage, and some are broader safety corpora rather than prompt-classifier benchmarks [1, 2, 5, 6].

From a benchmark-construction perspective, SHIBBOLETH adopts the view that the artifact should be treated as a governed dataset bundle rather than as an informal collection of CSV rows. This aligns with the dataset documentation and reproducibility discipline emphasized by datasheet-style governance work [3]. The additional requirement here is that reproducibility must hold across transformations, multilingual expansion, and evaluation-specific perturbations rather than only over a static released file.

The main gap this paper targets is not simply “more jailbreak data.” It is the lack of a benchmark grammar that combines multilingual coverage, leakage-resistant generation, and explicit accounting for utility and efficiency costs. In that sense, SHIBBOLETH is closer to an evaluation instrument than to a one-shot corpus release [4].

3 Threat Model

We assume a system in which a pre-LLM adversarial prompt classifier filters or routes user inputs before they reach a target language model. The classifier may allow, sanitize, step up, or block a request, and may be deployed in either stateless turn-level mode or stateful session-level mode. The surrounding evaluation harness treats the policy router and target-model interface as explicit parts of the measured system rather than hidden implementation details [4].

The adversary is assumed to have black-box access, repeated-query capability, multilingual prompting ability, code-mixing and Unicode obfuscation tools, and the ability to probe the system over multiple turns. The adversary does not have direct access to classifier weights or privileged runtime control.

We model two primary objectives:

Stage	Purpose
0. Fetch	pin source snapshots and checksums
1. Canonicalize	normalize text and assign stable source ids
2. Embed + cluster	construct semantic groups for leakage-safe splitting
3. Split	assign train/dev/test at cluster level
4. Tier-1 variants	generate high-priority multilingual variants
5. Tier-2 variants	extend language coverage under budget control
6. Eval suites	generate code-mix, homoglyph, and exhaustion masks
7. Verify	enforce invariants and write manifests

Table 1: The seven-stage SHIBBOLETH pipeline as a reproducible benchmark grammar.

1. **Under-blocking / safety failure.** The attacker causes adversarial prompts to pass through and reach the target model.
2. **Availability degradation / safety-DoS.** The attacker degrades service usefulness through benign over-blocking, refusal spirals, token burn, or latency inflation.

This framing is important because a classifier that blocks more attacks by catastrophically harming benign traffic is not obviously production-worthy. For that reason, the paper should treat adversarial prompt classification as a tri-objective evaluation problem rather than a single scalar accuracy problem.

The concrete attack families in scope include jailbreak attempts, prompt injection, instruction override, system-prompt extraction, tool misuse or exfiltration attempts, policy probing, multilingual evasion, code-mixed evasion, Unicode or homoglyph masking, and deliberate resource-exhaustion prompts. Just as importantly, the benchmark must include benign lookalikes such as quoted adversarial text, security research discussion, and multilingual content that resembles attacks superficially but should not be blocked. This separation between attack shape and classifier action is what makes over-blocking measurable instead of anecdotal.

4 Dataset and Pipeline

SHIBBOLETH uses a staged dataset build pipeline designed to produce replayable artifacts rather than ad hoc CSV snapshots. The current repository framing defines seven stages: fetch, canonicalize, embed and cluster, split, tier-1 multilingual generation, tier-2 multilingual generation, evaluation-suite transforms, and integrity verification [4].

The key invariant is *split before transform*. Canonical prompts are clustered and assigned to train, development, and evaluation partitions before translation or robustness transformations are applied. Every derived variant retains a pointer back to its originating source prompt. This design is intended to prevent semantic leakage across splits, especially in multilingual settings where translation can otherwise create near-duplicate evaluation items.

The pipeline also treats transforms as first-class objects rather than incidental scripts. Machine translation, code-mixing, homoglyph substitution, and exhaustion-style perturbations should all be generated with explicit configuration, seeded randomness where appropriate, and recorded provenance. This makes the benchmark grammar extensible: new transforms or attack families can be registered without rewriting the entire build system.

At the schema level, the benchmark is intentionally split into canonical prompts, derived variants, and eventually session traces. This keeps provenance explicit and makes it possible to compare single-

Partition	Primary failure mode	Main metrics
Benign lookalikes	over-blocking	BRR, latency
Direct attacks	under-blocking	APTR, AUROC
Obfuscation	robustness failure	RADR, PR-AUC
Exhaustion	availability loss	TBR, p50/p95, recovery time

Table 2: Evaluation partitions and their primary reporting targets.

turn and multi-turn evaluations without collapsing everything into one overloaded table. It also aligns the benchmark with broader dataset-governance expectations that emphasize documentation, lineage, and reproducibility rather than only raw scale [3].

For the paper, the remaining implementation work is to connect this declarative framing to a frozen build artifact and an auditable manifest chain. Once that exists, the paper can report not only the dataset contents but also the invariants that guarantee how those contents were produced.

5 Evaluation Framework

The evaluation plan is organized around four partitions: benign-but-high-risk-looking inputs, direct adversarial attempts, obfuscation and evasion attacks, and resource-exhaustion or safety-saturation workloads. This partitioning makes the paper’s reporting surface much clearer than a flat mixed test set.

The primary metrics are:

- attack pass-through rate for direct compromise risk,
- benign refusal rate for over-blocking,
- robust attack detection rate for transformed attacks, and
- token burn and latency overhead for availability impact.

The repo already frames these as Security–Utility–Efficiency tradeoffs. In paper form, that suggests reporting per-cell results over a matrix of policy layer, target model, language suite, and attack suite, then plotting Pareto fronts rather than forcing all policy configurations into one scalar leaderboard.

The harness design also implies a clean experimental matrix: policy layer, target model, language suite, attack suite, and later session template. This matters because classifier quality alone is not the whole story. A prompt classifier can appear strong while its surrounding policy logic quietly changes prompt structure, adds hidden instructions, or shifts cost and latency to the downstream model [4].

The missing empirical work is straightforward to name even if it is not complete yet: baseline rule systems, a lightweight learned classifier, policy router variants, multilingual robustness sweeps, and significance testing for the main table claims. Those experiments should be enough to turn the current design into a real benchmark paper rather than a design note.

6 Limitations and Future Work

The current benchmark design still has real limitations. Native adversarial coverage outside English remains sparse, so some language coverage will initially depend on translation and deterministic

masking rather than organically collected attacks. Phase 1 of the benchmark is also primarily single-turn, which means gradual multi-turn prompt injection remains only partially captured until the session pipeline lands.

There are also interface confounds. Different providers expose different token accounting, hidden routing behavior, and policy surfaces. If these are not normalized carefully, the benchmark can accidentally measure provider wrappers instead of prompt-classifier behavior. Finally, adversarial prompt styles change quickly. A benchmark grammar is more resilient than a frozen CSV, but temporal drift remains part of the problem and should be acknowledged explicitly.

7 Conclusion

SHIBBOLETH is most compelling when treated as a reproducible evaluation instrument instead of a one-off multilingual prompt dataset. The benchmark contribution is the grammar: a threat model, a leakage-resistant build discipline, and a tradeoff-aware evaluation harness that makes front-door prompt classification claims auditable.

The next milestone for this paper is empirical consolidation rather than conceptual expansion. Once the evaluation harness is frozen and the first end-to-end results land, the current draft should be able to support a benchmark paper with a clear production-facing thesis: adversarial prompt classifier evaluation should measure not only whether a wall blocks, but whether it blocks fairly, efficiently, and reproducibly.

References

- [1] All-Prompt-Jailbreak. All-prompt-jailbreak, 2024. Public jailbreak prompt corpus.
- [2] Aya RedTeaming. Aya redteaming, 2024. Multilingual red-teaming dataset.
- [3] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [4] Octave Commons. Shibboleth internal research draft and pipeline specifications, 2026. Repository-local working documents.
- [5] WildJailbreak. Wildjailbreak, 2024. Adversarial prompt benchmark and dataset.
- [6] XSafety. Xsafety, 2024. Multilingual safety dataset.